

Junior Software Engineer — Project Round

LMS (Learning Management System)

You've cleared the contest round and reached this stage. This is the most important step in the selection process — candidates for the final interview will be chosen from here.

In this round you'll build an **LMS (Learning Management System)**. Everyone gets the same project spec, so we can evaluate all submissions against the same bar.



Deadline

- **Handed out:** 24 August 2026
- **Deadline:** 30 August 2026, 11:59 PM

Submissions after the deadline will not be accepted. The submission form closes at 11:59 PM on 30 August 2026.



Tech Stack (mandatory — no deviations)

Layer	Technology	Hosting
Frontend	Next.js	Vercel
Backend / CMS	Strapi	Railway

You'll work on exactly this stack on the job — that's why it's fixed. Using any other framework or hosting means your submission is void.



User Roles

There are **four roles**, each with different permissions:

- **Admin** — full control of the platform. Manages users and assigns/changes their roles. Can do everything.

- **Content Manager** — creates and manages courses and lessons (the content library). Does **not** manage users.
- **Instructor** — manages the lessons and quizzes of their own courses, and can see the progress of students enrolled in them.
- **Student** — enrolls in courses, views lessons, takes quizzes, and tracks their own progress.

Access must differ strictly by role. A logged-in user must only be able to do what their role allows — nothing more.

Permission Matrix

Action	Admin	Content Manager	Instructor	Student
Manage users & assign roles	✓	✗	✗	✗
Create / edit / delete any course	✓	✓	Own only	✗
Add / edit / delete lessons	✓	✓	Own courses	✗
Create quizzes	✓	✓	Own courses	✗
View student progress	✓	✓	Own courses	Own only
Write / manage blog posts	✓	✓	✗	✗
Enroll in a course	✗	✗	✗	✓
Take quizzes	✗	✗	✗	✓

Getting this 4-role access control right — cleanly, without leaks — is itself part of what we're evaluating.

✓ Core Features (these must be present)

1. Authentication + Role-based access

- Sign up / Login, with a role for each user (admin / content manager / instructor / student).
- Role-based protected routes — a logged-out user or a user without the right role must not be able to access a restricted page or perform a restricted action. Enforce this on the backend, not just by hiding buttons.

2. Course Management (Admin / Content Manager / Instructor)

- Courses can be created / edited / deleted per the permission matrix above (Content Manager across the platform, Instructor for their own).
- Under each course, multiple **lessons** can be added / edited / deleted (title + content; content can be text or a video URL).

3. Course Enrollment (Student)

- Student can browse available courses and enroll.
- Enrolled courses show up separately under "My Courses".

4. Lesson Viewing (Student)

- Student can view the lessons of enrolled courses in sequence.

☀ Differentiator Features (this is where you stand out)

Most candidates can deliver the core features. **The real differentiation happens on these features** — do them well, with edge cases handled, and you'll be ahead.

1. Progress Tracking

- When a student finishes a lesson, they can mark it "complete".
- For each course, show the student's **progress percentage** (e.g. 3 of 5 lessons done = 60%).
- This progress must be accurate per student, per course — and must persist across refreshes.

2. Quiz with Auto-Grading

- An instructor / content manager can add an MCQ quiz to a course (question + options + correct answer).
- Student takes the quiz and gets an **automatic score** immediately on submit.
- The student's quiz result is stored and viewable later.

3. Admin Panel

- A dedicated admin dashboard, accessible only to the **admin** role.
- Admin can see all users and **manage their roles** (promote / change / remove a user's role).
- Admin can view and manage all courses, lessons, and blog posts across the platform.
- Show some basic platform stats (e.g. total users per role, total courses, total enrollments).

4. Blog — Writing & Control

- Content Manager (and Admin) can **write, edit, publish, and delete blog posts** (title + body; a cover image URL is fine).
- Draft vs Published state — only published posts are visible to students / public; drafts are not.
- Anyone can read the published blog list and open a single post.
- Admin has full control over every blog post (including others'); Content Manager manages the posts they can create per the matrix.

Video Walkthrough (mandatory — 10 minutes)

You must record a video of **up to 10 minutes** where you explain the project yourself. This is what we'll review most closely — being able to explain your own code is the core test of this round.

The video must cover each of the following:

Live demo — walk through the full flow on the deployed app across the roles: student (enroll → lesson → progress → quiz), instructor / content manager (create course → lesson → quiz → blog post), and admin (admin panel → manage a user's role).

Data flow — pick one feature and show how data moves from the frontend to the Strapi backend and back.

Role-based access — briefly show how you enforce permissions on the backend (not just by hiding UI).

Progress tracking logic — how you implemented it, where and how the data is stored, explained line by line.

Quiz auto-grading logic — how the score is computed, shown in code.

Admin panel + blog — show the admin managing roles, and the blog draft → publish flow.

Deployment setup — briefly show how you configured Vercel and Railway and how you handled environment variables.

⚠️ If you can't explain your own code, you're out. The video must be a screen recording, in your own voice.



Submission

When you're done, submit the following **in one form**:

1. **GitHub repository link** (public — both frontend + backend)
2. **Frontend URL** — the live Vercel link
3. **Backend URL** — the live Railway link
4. **Video walkthrough link** (Google Drive / YouTube unlisted — must be openable)

If any one of these is missing, the submission is treated as incomplete.



Important Rules

- The deployed app must stay **live** at least until the interviews are over.
 - The GitHub repo must have a proper **commit history** — pushing the whole project in a single commit is a negative signal.
 - In the README, briefly write: how to run it locally, and which features you completed.
-



On Using AI

You are **free to use any AI tool** you want — that's not against the rules, and we're not here to police it.

But understand clearly: **a fully AI-generated project will not be accepted**. If your own thinking and creativity don't show up anywhere in the work, you will be rejected — no matter how "complete" the project looks.

We want to see *you* in the project: the decisions you made, the way you structured things, the parts you built and understood yourself. The video walkthrough is where this becomes obvious

— someone who leaned on AI for everything won't be able to explain their own code, and that's exactly what we're looking for.

Use AI as a tool. Don't let it replace your thinking.

Good luck. Whatever you build, build it understanding it — that's what we want to see.